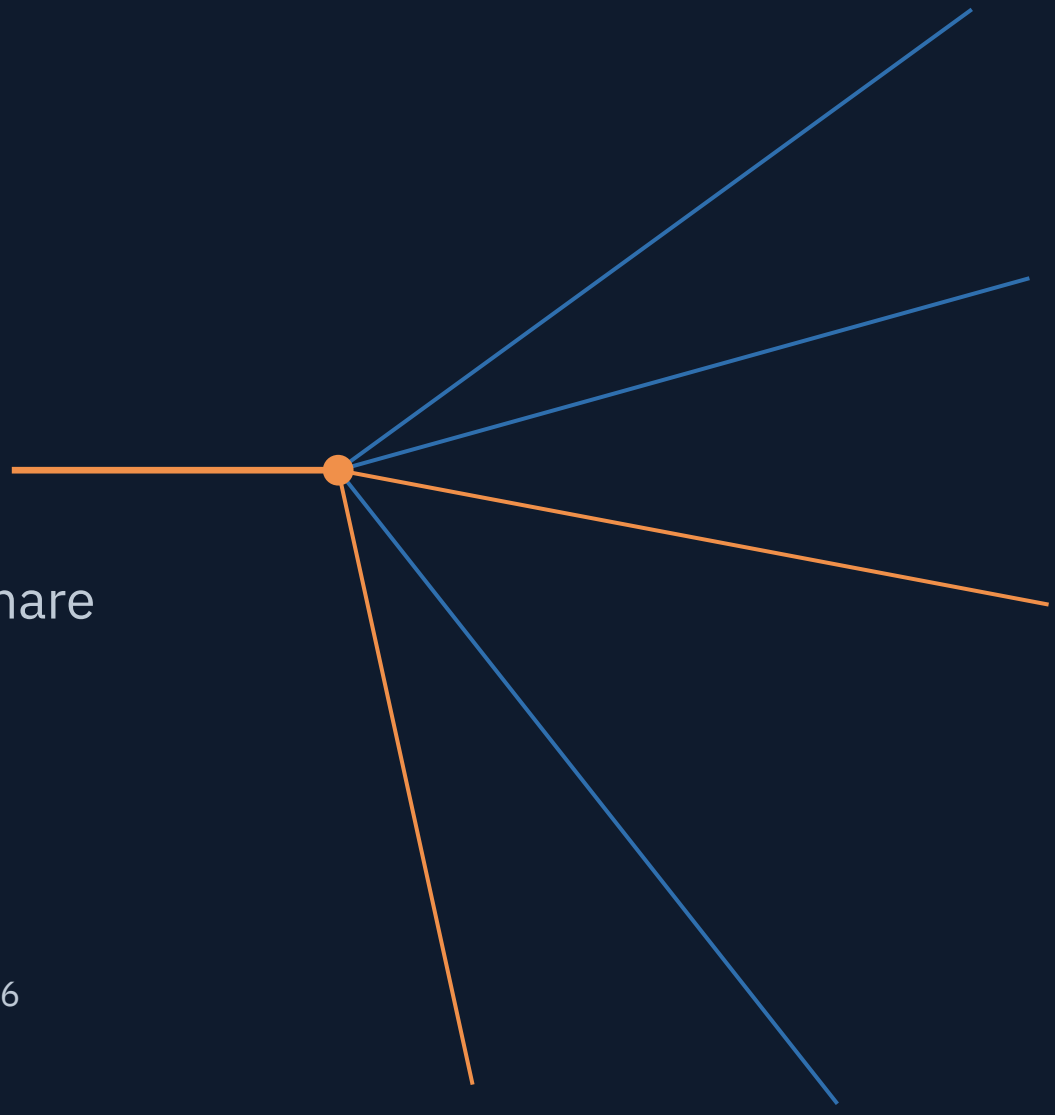
PARTICLE DETECTOR DESIGN AND SIMULATIONS

# Coding with Geant4



Writing a first simulation, the Bragg peak and process-share examples

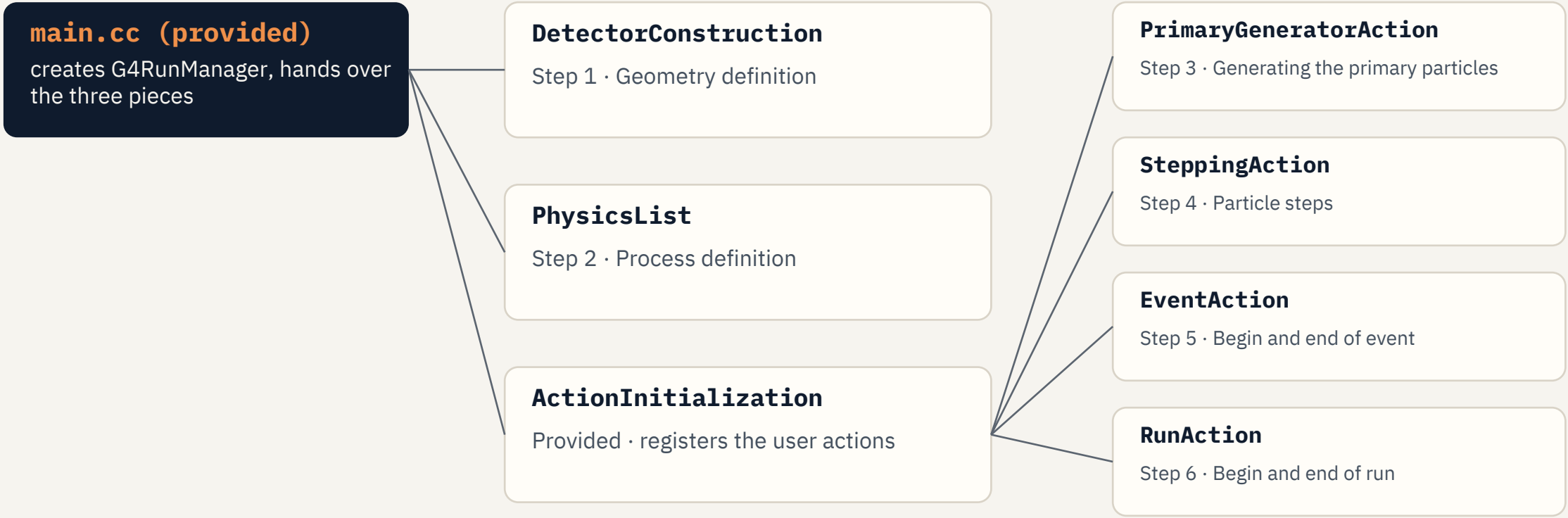
Kaan Yüksel OYULMAZ · Ankara University Institute of Accelerator Technologies · 2026

CODING WITH GEANT4

# What will we do in this lecture?

| | | |
|---|---|---|
| 00 | **How do the classes work?** | We fill in, Geant4 calls; run / event / step |
| 01 | **Geometry** | DetectorConstruction: Vacuum world + 300 µm Si slab |
| 02 | **Physics list** | PhysicsList: Which processes are simulated |
| 03 | **Particle source** | PrimaryGeneratorAction: G4ParticleGun, $\mu^-$ 1 GeV |
| 04 | **SteppingAction** | In the slab? If so, add the deposited energy to the event |
| 05 | **EventAction** | Total energy per event → fill the histogram when the event ends |
| 06 | **RunAction** | Histogram definition; open / write / close the file |
| 07 | **Result** | Analysis with ROOT: Landau distribution |
| 08 | **Bragg peak** | On the same code: Water, 20 cm long, proton source; Bragg peak histogram |
| 09 | **Share of processes** | Random energy; creator process of secondaries; ionisation / bremsstrahlung / pair |

# How do the classes work?




**Key idea:** We fill in the blocks, Geant4 calls them. For each step we will write the body of one file.

# When is it called? Run, event, step

**Run**

/run/beamOn 10000

**RunAction**

`BeginOfRunAction` · `EndOfRunAction`

once: Open the file; at the end write and close

**Event**

1 muon + everything it produces in the world

**EventAction**

`BeginOfEventAction` · `EndOfEventAction`

10 000 times: fEdep = 0; at the end FillH1(0, fEdep)

**Step**

between two interactions / boundaries

**SteppingAction**

`UserSteppingAction`

hundreds of thousands of times: Add edep to the event

> **Data-collection rule:** Measure in the step, sum in the event, write in the run.

# Empty Geant4 lecture skeleton: G4Ders

**file tree (orange: filled in during the lecture)**

```
G4Ders/
  CMakeLists.txt
  main.cc                  # provided
  macros/mip_si.mac        # provided
  include/                 # 7 headers
  src/
    ActionInitialization.cc   # provided
    DetectorConstruction.cc   # step 1
    PhysicsList.cc            # step 2
    PrimaryGeneratorAction.cc # step 3
    SteppingAction.cc         # step 4
    EventAction.cc            # step 5
    RunAction.cc              # step 6
```

**build and run (WSL)**

```
source /opt/geant4/bin/geant4.sh
cd G4Ders
mkdir build
cd build
cmake ../.
make -j4
```

- **Empty bodies:** Every function is just { }
- **Headers provided:** Member variables and function signatures are written; we fill in the bodies.
- **While empty:** It compiles without errors; when run it crashes because there is no world (Segmentation fault). First job: Fill in Construct().
- **After every change:** make –j4; ./ders mip_si.mac

# Three-layer geometry: Solid, logical, physical

### 1 · Solid — G4Box

Shape and size. Half-lengths are given: Dz = 150 µm for a 300 µm thickness.

```
new G4Box("Silicon", 5*cm, 5*cm, 150*um)
```

### 2 · Logical volume — G4LogicalVolume

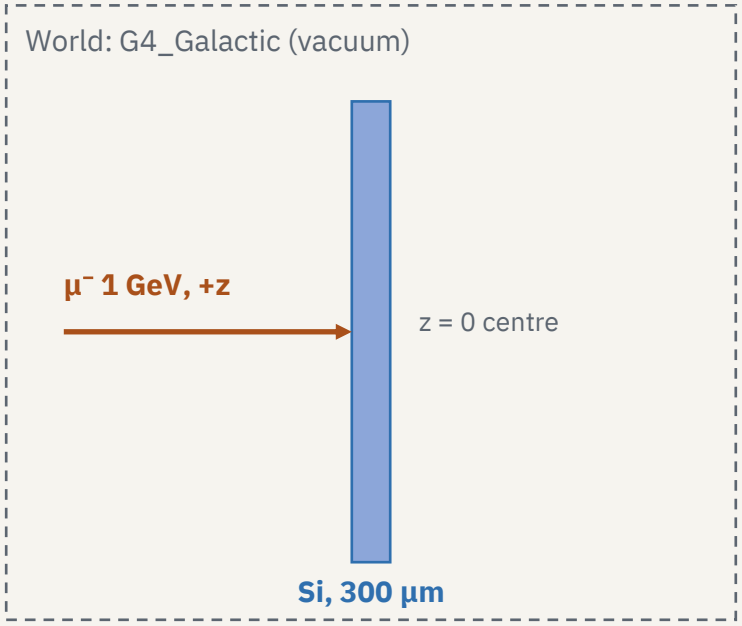
Shape + material.

```
new G4LogicalVolume(solidSi, silicon, "Silicon")
```

### 3 · Physical volume — G4PVPlacement

Places the logical volume inside a mother volume, at a position.

```
new G4PVPlacement(nullptr, G4ThreeVector(), logicSi, "Silicon", logicWorld, false, 0)
```




**Material: By NIST name: G4_Si, G4_Galactic**
**World** is the outermost volume; Construct returns it.

# DetectorConstruction::Construct()

```
G4VPhysicalVolume* DetectorConstruction::Construct()
{
  // 1) materials (by NIST name)
  G4NistManager* nist = G4NistManager::Instance();
  G4Material* vacuum  = nist->FindOrBuildMaterial("G4_Galactic");
  G4Material* silicon = nist->FindOrBuildMaterial("G4_Si");

  // 2) World: a box filled with vacuum (half-lengths!)
  G4Box* solidWorld = new G4Box("World", 10*cm, 10*cm, 10*cm);
  G4LogicalVolume* logicWorld = new G4LogicalVolume(solidWorld, vacuum, "World");
  G4VPhysicalVolume* physWorld = new G4PVPlacement(nullptr, G4ThreeVector(),
                                   logicWorld, "World", nullptr, false, 0);

  // 3) Slab: at the centre, 300 um silicon (half-thickness 150 um)
  G4Box* solidSi = new G4Box("Silicon", 5*cm, 5*cm, 150*um);
  G4LogicalVolume* logicSi = new G4LogicalVolume(solidSi, silicon, "Silicon");
  new G4PVPlacement(nullptr, G4ThreeVector(), logicSi, "Silicon",
                    logicWorld, false, 0);

  return physWorld;   // 4) the world is returned
}
```

- **Units:** Every length is multiplied by a unit: 150*um, 5*cm. A number without a unit is taken as mm!
- **Half-lengths:** G4Box wants half-lengths, not full lengths: 150*um for 300 µm.
- **G4ThreeVector():** The position (0,0,0); slab and world share the same centre.
- **Names:** "Silicon" is the name of the physical volume; in step 4 the question "am I in the slab?" will be asked with this name.

**Checkpoint:** Build, ./ders mip_si.mac → no crash any more; the geometry is built, but there is no particle source and beamOn fails because the physics list is not defined yet.

# Where to look: Shapes and materials

## Shapes (solids)

| Class | Shape and parameters |
|---|---|
| G4Box | box: half-lengths dx, dy, dz |
| G4Tubs | cylinder / tube: inner-outer radius, half-length, angle |
| G4Cons | cone: Radii at both ends, half-length |
| G4Sphere | sphere / shell: Radii, angles |
| G4Trd | trapezoid: Dx, dy at both ends; dz |
| G4UnionSolid | Boolean: Union, subtraction, intersection |

```
new G4Tubs("Pipe", 0, 2*cm, 10*cm, 0, 360*deg)
```

**Docs:** Application Developer Guide → Detector → Geometry → Solids
https://geant4.web.cern.ch/documentation/pipelines/master/bfad_html/ForApplicationDevelopers/Detector/Geometry/geomSolids.html

## Materials (NIST names)

| Name | Material |
|---|---|
| G4_Si, G4_Al, G4_Pb | elements: G4_ + symbol |
| G4_WATER, G4_AIR | compound / mixture |
| G4_PLASTIC_SC_VINYLTOLUENE | plastic scintillator |
| G4_SODIUM_IODIDE | NaI crystal |
| G4_Galactic | vacuum (for the world) |
| G4_lAr, G4_Ge | liquid argon, germanium |

```
/material/nist/listMaterials      # full list to the terminal
```

**Docs:** Application Developer Guide → Appendix → NIST Material Names
https://geant4.web.cern.ch/documentation/dev/bfad_html/ForApplicationDevelopers/Appendix/materialNames.html

> **Habit:** Whenever you need a new shape or material, look at these two pages first!

# PhysicsList: Which processes are simulated?

```
// PhysicsList.cc: ready-made building blocks are registered
PhysicsList::PhysicsList()
{
  RegisterPhysics(new G4EmStandardPhysics_option4()); // electromagnetic
  RegisterPhysics(new G4DecayPhysics());              // decays
  RegisterPhysics(new G4HadronElasticPhysics());      // hadron elastic
  RegisterPhysics(new G4HadronPhysicsFTFP_BERT());    // nuclear
  RegisterPhysics(new G4IonPhysics());                // alpha, ions

  SetDefaultCutValue(0.1 * mm);                       // production cut
}
```

| Particle | Process name | Physics (last lecture) |
|---|---|---|
| μ, p, α | `muIoni, hIoni, ionIoni` | ionisation: Bethe–Bloch |
| $e^-$, $e^+$ | `eIoni, eBrem` | ionisation, bremsstrahlung |
| all charged | `msc` | multiple Coulomb scattering |

- **Separate class:** PhysicsList : G4VModularPhysicsList. RegisterPhysics in the constructor.
- **option4:** The most accurate EM physics. Required for a correct dE/dx in a thin layer; today's measurement relies on it.
- **Used today:** Only the EM part (muon ionisation). The rest is for later lectures.
- **Cut 0.1 mm:** ≈ 80 keV in silicon: δ-electrons below this are not tracked separately; their energy is deposited locally.
- **in main.cc (provided):** runManager->SetUserInitialization(new PhysicsList());

**Checkpoint:** Build, run → run initialize sets up geometry + physics; beamOn now works but no particle is generated (no source).

# PrimaryGeneratorAction: G4ParticleGun

```
PrimaryGeneratorAction::PrimaryGeneratorAction()
{
  fGun = new G4ParticleGun(1);   // 1 particle per event

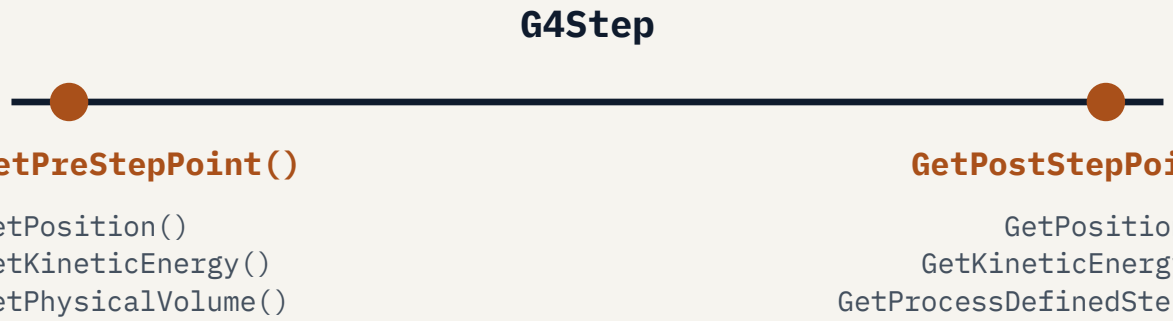
  G4ParticleDefinition* particle =
      G4ParticleTable::GetParticleTable()->FindParticle("mu-");
  fGun->SetParticleDefinition(particle);
  fGun->SetParticleEnergy(1 * GeV);
  fGun->SetParticleMomentumDirection(G4ThreeVector(0., 0., 1.));  // +z
  fGun->SetParticlePosition(G4ThreeVector(0., 0., -1 * cm));  // in front
}

void PrimaryGeneratorAction::GeneratePrimaries(G4Event* event)
{
  fGun->GeneratePrimaryVertex(event);   // add the particle to the event
}
```

- **G4ParticleGun:** The simplest source: a single particle, fixed position and direction.
- **FindParticle("mu-"):** From the particle table by name: e-, proton, alpha, gamma...
- **Position:** z = −1 cm; the world is vacuum, the muon reaches the slab with exactly 1 GeV.
- **Built-in commands:** /gun/particle, /gun/energy; changed from the macro.
- **GeneratePrimaryVertex:** if forgotten, the event is empty and no error is given.

**Checkpoint:** Build, ./ders mip_si.mac → 10 000 muons cross the slab, the run finishes in the terminal. No measurement yet.

# What is inside a G4Step?




**The step itself:** `GetTotalEnergyDeposit()` the energy deposited in this step — today's only measurement.
**The track taking the step:** `GetTrack()` → G4Track: `GetVolume()->GetName()`, `GetTrackID()`, `GetKineticEnergy()`, `GetDefinition()` …

- **Two questions today:** (1) is the step in the slab? The name of the volume the track is in: GetTrack()->GetVolume()->GetName(). (2) How much energy was deposited? GetTotalEnergyDeposit().
- **Energy deposit:** a property of the step, not of a point.
- **Muon in 300 µm Si:** ≈ 6 steps, 10–30 keV each; ≈ 100 keV per event in total.
- **In the next examples:** Secondaries, depth calculation and processes.

# SteppingAction::UserSteppingAction()

```
void SteppingAction::UserSteppingAction(const G4Step* step)
{
  // 1) volume name: step -> track -> volume -> name
  G4String volumeName = step->GetTrack()->GetVolume()->GetName();
  if (volumeName != "Silicon") return;   // not the slab: ignore

  // 2) energy deposited in this step -> add to the event
  G4double edep = step->GetTotalEnergyDeposit();
  fEvent->AddEdep(edep);
}
```

```
// EventAction.hh (provided):
void AddEdep(G4double e) { fEdep += e; }   // called every step
```

- **Called every step:** Hundreds of thousands of times; it only measures and passes to EventAction (as in example B1).
- **GetTrack():** The track taking the step (G4Track). Volume, energy, track ID, particle type: All reached through this door.
- **GetVolume()->GetName():** Name of the physical volume where the step started; in step 1 we called it "Silicon". In the world it returns "World" and we leave.
- **No histogram:** Measure in the step, sum in the event, write in the run.

**Checkpoint:** Build, run → nothing visible changes: Energy is being summed but not yet written anywhere.

# EventAction: Summing the energy per event

```
void EventAction::BeginOfEventAction(const G4Event*)
{
  fEdep = 0.;   // new event: reset the sum
}

void EventAction::EndOfEventAction(const G4Event*)
{
  G4AnalysisManager::Instance()->FillH1(0, fEdep);   // write to the
histogram
}
```

```
// EventAction.hh (provided):
void AddEdep(G4double e) { fEdep += e; }
private:
  G4double fEdep = 0.;   // total energy deposited in the slab in this event
```

- **Reset at the start, write at the end:** One variable per event: fEdep.
- **FillH1(0, fEdep):** 0 = histogram id (defined in step 6); the value is in internal units (MeV).
- **G4AnalysisManager:** Geant4's bookkeeping tool for analysis; a single object, reached with Instance().
- **One entry per event:** 10 000 events → 10 000 entries → a distribution.

**Checkpoint:** Build, run → no ROOT output yet: The histogram is not defined, so no file was opened.

# RunAction: Histogram and file

```
RunAction::RunAction()
{
  G4AnalysisManager* am = G4AnalysisManager::Instance();
  am->SetDefaultFileType("root");
  am->SetFileName("ders");
  am->CreateH1("edep", "Energy deposited per event",
               200, 0., 500.*keV, "keV");
}

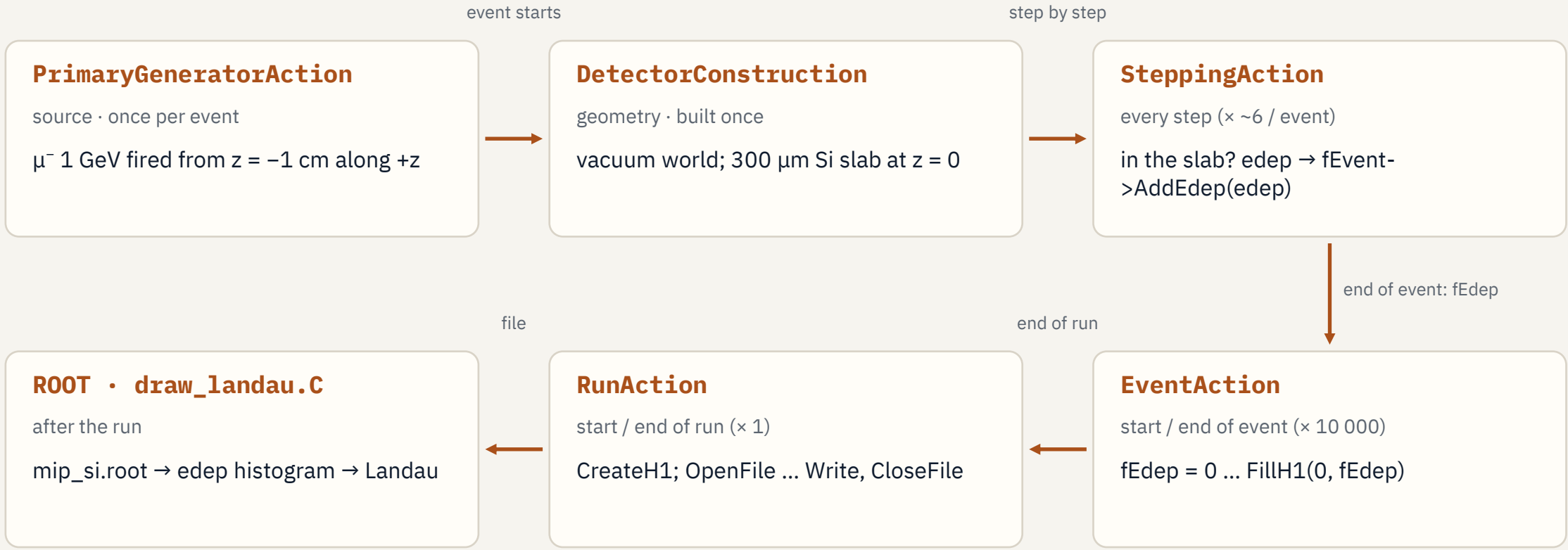
void RunAction::BeginOfRunAction(const G4Run*)
{
  G4AnalysisManager::Instance()->OpenFile();   // open the file
}

void RunAction::EndOfRunAction(const G4Run* run)
{
  G4AnalysisManager* am = G4AnalysisManager::Instance();
  am->Write();       // write the histogram
  am->CloseFile();   // close the file
  G4cout << "Number of events: " << run->GetNumberOfEvent() << G4endl;
}
```

- **Histogram id:** Order of definition; the first CreateH1 → 0. EventAction's FillH1(0, ...) writes here.
- **Units:** Limits in internal units: 500.*keV. "keV" only labels the axis; FillH1 also receives internal units (MeV).
- **OpenFile / Write / CloseFile:** Start / end of run. If CloseFile is forgotten the file is empty.
- **File name:** From the macro: /analysis/setFileName mip_si → mip_si.root.

**Checkpoint:** Build, ./ders mip_si.mac → "Number of events: 10000" and mip_si.root. Measurement done.

# What did we do? Flow of the code




**Rule:** Measure in the step (SteppingAction), sum in the event (EventAction), write in the run (RunAction).

CODING WITH GEANT4 · 07 RESULT

# Running with a macro

**macro**

```
# mip_si.mac
/run/initialize           # builds geometry + physics

/gun/particle mu-
/gun/energy 1 GeV

/analysis/setFileName mip_si
/run/printProgress 1000
/run/beamOn 10000         # run: 10 000 events
```

**build and run (WSL)**

```
Cd G4Ders
mkdir build
Cd build
cmake ../.
make –j4
./ders mip_si.mac
root -l draw_landau.C
```

- **Order:** /gun/ commands come after /run/initialize.
- **Another particle:** /gun/particle proton, /gun/energy 100 MeV — the code does not change.
- **./ders without arguments:** Graphical interface; volumes and tracks are shown (VcXsrv must be running).

# Analysis with ROOT: draw_landau.C

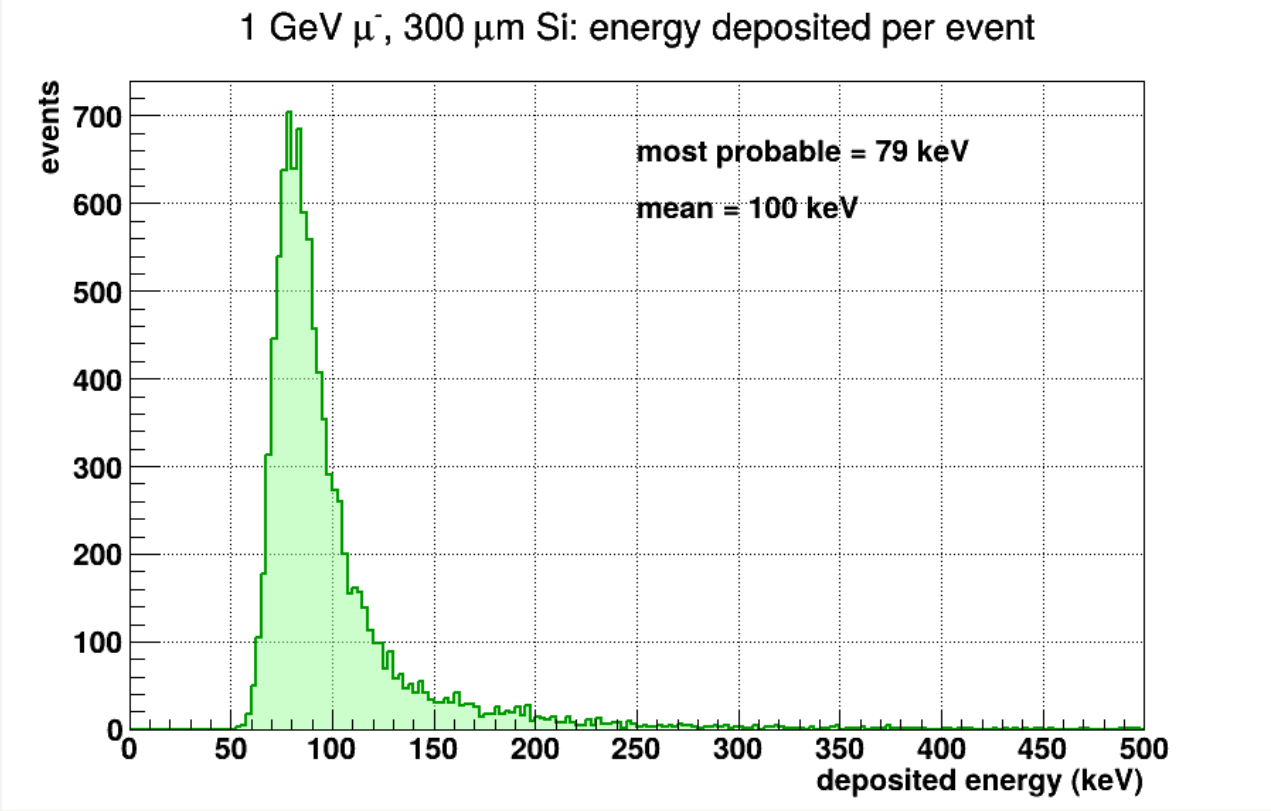
```
// draw_landau.C — draws the edep histogram in mip_si.root
void draw_landau()
{
  TFile* f = TFile::Open("mip_si.root");
  TH1* h = (TH1*)f->Get("edep");

  TCanvas* c = new TCanvas("c", "", 900, 600);
  h->SetTitle("1 GeV mu-, 300 um Si;deposited energy (keV);events");
  h->SetLineColor(kGreen + 2);
  h->Draw("hist");

  double mpv  = h->GetBinCenter(h->GetMaximumBin());   // MPV
  double mean = h->GetMean();                          // mean
  printf("most probable = %.0f keV, mean = %.0f keV\n", mpv, mean);
  c->SaveAs("landau.png");
}
```

- **TFile::Open / Get:** Open the file, get the histogram by name ("edep").
- **Draw("hist"):** Draw as a line.
- **Most probable vs mean:** Position of the peak and the arithmetic mean; different for a Landau.
- **By hand as well: root -l mip_si.root → edep->Draw()**

# Result: Landau distribution




- **Why asymmetric?** few collisions in a thin slab; rare large transfers (δ-rays) make the tail on the right. Most probable < mean.
- **Try:** /gun/energy 10 MeV or /gun/particle proton: How does the distribution change?

# Example: Bragg curve

**1** **DetectorConstruction**

G4_Si → G4_WATER; 300 µm → 20 cm, centred. Source moved 10 cm back: z = −11 cm

**2** **SteppingAction**

depth of the step z + 10 cm; fEvent->AddDepth(z, edep)

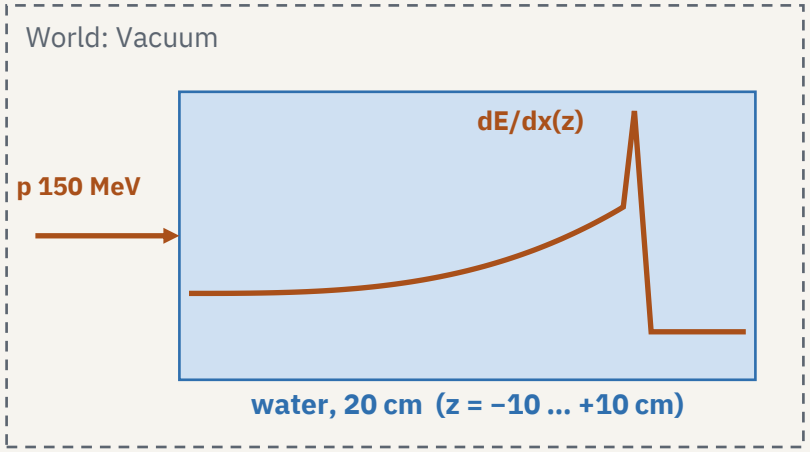
**3** **RunAction**

ikinci histogram: CreateH1("depth", ..., 0., 20.*cm, "cm") → id 1

**4** **EventAction**

AddDepth(z, edep): FillH1(1, z, edep); weight = energy

Unchanged: PhysicsList, main.cc. Only the position in PrimaryGeneratorAction. Macro: /gun/particle proton, /gun/energy 150 MeV.




**Depth histogram:** Every step fills x = z, **weight = edep**. Bin content = energy deposited at that depth.
**1 mm bins, 1000 protons:** Dividing by 1000 makes the y axis MeV/mm, i.e. dE/dx.
**Expectation (last lecture):** The range of a 150 MeV proton in water is ≈ 15.8 cm; the peak is there.

# Change 1: DetectorConstruction, water block

```
G4VPhysicalVolume* DetectorConstruction::Construct()
{
  G4NistManager* nist = G4NistManager::Instance();
  G4Material* vacuum = nist->FindOrBuildMaterial("G4_Galactic");
  G4Material* water  = nist->FindOrBuildMaterial("G4_WATER");
  G4Box* solidWorld = new G4Box("World", 25*cm, 25*cm, 25*cm);  // bigger
  G4LogicalVolume* logicWorld = new G4LogicalVolume(solidWorld, vacuum, "World");
  G4VPhysicalVolume* physWorld = new G4PVPlacement(nullptr, G4ThreeVector(),
                                   logicWorld, "World", nullptr, false, 0);
  // water block: centred, 20 cm long (half 10 cm): z = -10 ... +10 cm
  G4Box* solidWater = new G4Box("Water", 5*cm, 5*cm, 10*cm);
  G4LogicalVolume* logicWater = new G4LogicalVolume(solidWater, water, "Water");
  new G4PVPlacement(nullptr, G4ThreeVector(), logicWater, "Water",
                    logicWorld, false, 0);
  // colour: blue, semi-transparent, solid drawing  (#include "G4VisAttributes.hh")
  G4VisAttributes* blue = new G4VisAttributes(G4Colour(0.2, 0.5, 1.0, 0.4));
  blue->SetForceSolid(true);
  logicWater->SetVisAttributes(blue);
  return physWorld;
}
```

- **Orange lines:** The changes; the rest is unchanged.
- **G4_WATER:** NIST name; 1 g/cm³.
- **Half-length 10 cm:** Block 20 cm; world 25 cm so the block stays inside.
- **Centred:** G4ThreeVector(); z = −10 … +10 cm.
- **Source moved back:** z = −11 cm in PrimaryGeneratorAction; otherwise born inside the water.
- **Colour:** G4Colour(r, g, b, opacity), each 0–1; SetForceSolid fills. Appearance only.

```
// PrimaryGeneratorAction.cc, constructor
fGun->SetParticlePosition(
    G4ThreeVector(0., 0., -11*cm));
```

**Checkpoint:** Build, ./ders bragg.mac → 1000 protons stop in the water; the edep histogram overflows (150 MeV), not important. No depth yet.

# Change 2: SteppingAction, depth calculation

```
void SteppingAction::UserSteppingAction(const G4Step* step)
{
  G4String volumeName = step->GetTrack()->GetVolume()->GetName();
  if (volumeName != "Water") return;   // the name changed

  G4double edep = step->GetTotalEnergyDeposit();
  fEvent->AddEdep(edep);

  // mid-point; water starts at z = -10 cm -> depth = z + 10 cm
  G4double z1 = step->GetPreStepPoint()->GetPosition().z();
  G4double z2 = step->GetPostStepPoint()->GetPosition().z();
  G4double z = 0.5 * (z1 + z2) + 10*cm;
  fEvent->AddDepth(z, edep);
}
```

```
// EventAction.hh: signature of the new function (in the public part)
  void AddDepth(G4double z, G4double edep);
```

- **Name changed:** "Water" instead of "Silicon"; a single word, since we compare by name.
- **GetPosition():** Position of the step point; z is the depth direction.
- **Mid-point:** a step in water is a few mm; writing the energy at its start makes a spike in the first bin.
- **+ 10 cm:** The block starts at −10 cm. G4SystemOfUnits.hh is needed.
- **AddDepth(z, edep):** As before, measuring in SteppingAction, writing in EventAction.
- **Header file:** It does not compile until the signature is added; otherwise the error "AddDepth is not a member".

**Checkpoint:** Does not compile yet: AddDepth has no body. Build after changes 3 and 4.

# Changes 3 and 4: Define and fill the histogram

```
// RunAction.cc, constructor: second histogram (id = 1)
  am->CreateH1("edep",  "Energy deposited per event",
               200, 0., 500.*keV, "keV");
  am->CreateH1("depth", "Energy deposited vs depth",
               200, 0., 20.*cm, "cm");   // bin = 1 mm
```

```
// EventAction.cc: SteppingAction calls this every step
void EventAction::AddDepth(G4double z, G4double edep)
{
  G4AnalysisManager::Instance()->FillH1(1, z, edep);   // weight = edep
}

// Begin/EndOfEventAction unchanged: fEdep = 0 ... FillH1(0, fEdep)
```

- **id = 1:** The second histogram defined. Must match FillH1(1, ...).
- **FillH1(id, x, weight):** x = z (internal unit mm, axis in cm); weight = deposited energy.
- **Without the weight:** It would count steps; we need energy.
- **Not at the end of the event:** Depth is written every step; the total energy at the end of the event.

**Checkpoint:** Build, ./ders bragg.mac → "Number of events: 1000"; bragg.root contains two histograms, edep and depth.

# Run and analyse: Bragg.mac, draw_bragg.C

**macro**

```
# bragg.mac
/run/initialize
/gun/particle proton
/gun/energy 150 MeV
/analysis/setFileName bragg
/run/printProgress 100
/run/beamOn 1000
```

**run**

```
source /opt/geant4/bin/geant4.sh
cp -r G4Ders G4Ders-bragg
cd G4Ders-bragg
mkdir build
cd build
cmake ../.
make -j4
./ders mip_si.mac
```

**draw_bragg.C**

```
void draw_bragg()
{
  TFile* f = TFile::Open("bragg.root");
  TH1* h = (TH1*)f->Get("depth");

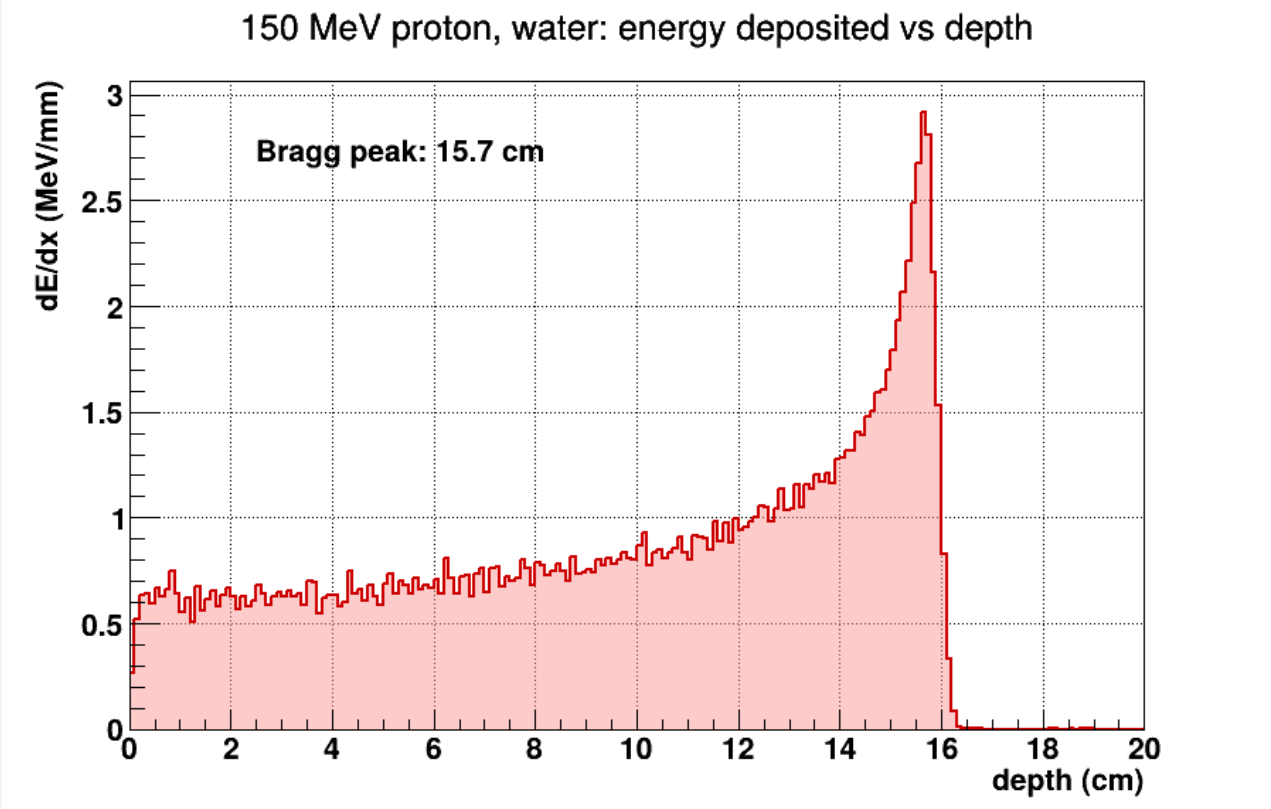
  h->Scale(1.0 / 1000);   // per proton; bin 1 mm -> MeV/mm

  TCanvas* c = new TCanvas("c", "", 900, 600);
  h->SetTitle("150 MeV proton, water;depth (cm);dE/dx (MeV/mm)");
  h->SetLineColor(kRed + 1);
  h->Draw("hist");

  double peak = h->GetBinCenter(h->GetMaximumBin());   // peak position
  printf("Bragg peak: %.1f cm\n", peak);
  c->SaveAs("bragg.png");
}
```

**1000 events are enough:** Each proton takes ≈ 360 steps; 360 000 entries in the histogram.

# Result: Bragg peak




- **Entrance ≈ 0.6 MeV/mm, peak ≈ 2.9 MeV/mm:** dE/dx rises as the proton slows down ($1/v^2$), and is largest just before it stops.
- **Peak at 15.7 cm:** NIST range 15.8 cm (150 MeV, water). Zero afterwards: The proton has stopped.
- **Difference from Landau:** There we summed per event; here every step is written at its depth, weighted by its energy.
- **Try:** /gun/energy 100 MeV → peak at 7.7 cm; 200 MeV → range 26 cm, the block is too short. /gun/particle e- → no peak.

**Same skeleton, four small changes:** From Landau to Bragg. Next: Which process takes the energy? Secondary particles and process names.

# Which process takes the energy?

1. **PrimaryGeneratorAction**
   random E0 every event: log10(E0/MeV) uniform in −1 … 7 (0.1 MeV … 10 TeV)

2. **SteppingAction**
   primary only (track 1): Continuous loss + process and energy of the secondaries born in this step

3. **EventAction**
   AddLoss(process, E0, energy): if → ionisation / bremsstrahlung / pair → FillH1(2|3|4, log10 E0, energy)

4. **RunAction**
   three histograms: Ioni, brem, pair; 16 bins, −1 … 7 (half a decade)

Unchanged: Geometry (20 cm water), PhysicsList, main.cc.
Three macros: /gun/particle e-, mu-, proton; 4000 events.

- **Question:** Through which process does a particle of energy E0 lose its energy in water?
- **Measurement idea:** When a secondary is born, Geant4 stores the name of the process that created it. It is reached with GetCreatorProcess(). We write its energy into that process's histogram.
- **Continuous loss:** Small transfers below the cut produce no secondary and arrive as edep. That is ionisation too.

# Change 1: Random energy in the source

```
#include "Randomize.hh"   // G4UniformRand

void PrimaryGeneratorAction::GeneratePrimaries(G4Event* event)
{
  // log10(E/MeV) uniform between -1 and 7: equal events per decade
  G4double logE = -1. + 8. * G4UniformRand();      // 0..1 -> -1..7
  G4double energy = std::pow(10., logE) * MeV;     // 0.1 MeV .. 10 TeV
  fGun->SetParticleEnergy(energy);

  fGun->GeneratePrimaryVertex(event);   // add the particle to the event
}
```

```
// SetParticleEnergy(1 * GeV) is removed from the constructor; energy is now
chosen every event
```

- **G4UniformRand():** Uniform random number between 0 and 1; Geant4's own generator.
- **Why the logarithm?** choosing E uniformly would put 90% of the events above 1 TeV. Uniform log E: Equal events per decade.
- **pow(10, logE):** From logarithm to energy; the unit is multiplied afterwards (MeV).
- **Every event:** GeneratePrimaries is called per event; the constructor only once. Anything random goes here.
- **Macro:** /gun/energy no longer has an effect; the particle type via /gun/particle.

**Checkpoint:** Build and run → every event has a different energy, but no process measurement yet.

# Change 2: SteppingAction, secondaries

```
  G4Track* track = step->GetTrack();   // (volume name, edep, depth: as before)
  // --- processes: primary particle only (track ID 1)
  if (track->GetTrackID() != 1) return;
  G4double e0 = track->GetVertexKineticEnergy();   // initial energy
  // 1) continuous loss along the step (below the cut): ionisation
  fEvent->AddLoss("Ioni", e0, edep);
  // 2) secondaries born in this step: which process took the energy?
  const std::vector<const G4Track*>* secondaries = step->GetSecondaryInCurrentStep();
  for (size_t i = 0; i < secondaries->size(); i++) {
    const G4Track* sec = (*secondaries)[i];
    G4String process = sec->GetCreatorProcess()->GetProcessName();
    G4double energy = sec->GetKineticEnergy();
    fEvent->AddLoss(process, e0, energy);
  }
}
```

```
  void AddLoss(G4String process, G4double e0, G4double energy);   // EventAction.hh
```

- **Track 1:** The primary particle. Counting the secondaries' steps would count the same energy twice.
- **GetVertexKineticEnergy():** Energy of the track at birth = E0; the x of the histogram.
- **Two losses:** Continuous (edep, below the cut) and given to a secondary (δ, photon, pair).
- **GetSecondaryInCurrentStep():** The list of secondaries born in this step; a pointer, dereferenced with * first.
- **GetCreatorProcess():** The process that created the secondary; its name: eIoni, eBrem, muPairProd...
- **Headers:** Add G4Track.hh and G4VProcess.hh.

**Checkpoint:** Does not compile yet: AddLoss has no body. Build after changes 3 and 4.

# Changes 3 and 4: Classify with if

```
// EventAction.cc: into one of three histograms by process name
void EventAction::AddLoss(G4String process, G4double e0, G4double energy)
{
  G4double x = std::log10(e0 / MeV);
  G4AnalysisManager* am = G4AnalysisManager::Instance();
  if (process == "Ioni" || process == "eIoni" ||
      process == "muIoni" || process == "hIoni") {
    am->FillH1(2, x, energy);   // ionisation (cont. loss + delta e-)
  }
  else if (process == "eBrem" || process == "muBrems" || process == "hBrems") {
    am->FillH1(3, x, energy);   // bremsstrahlung
  }
  else if (process == "muPairProd" || process == "hPairProd") {
    am->FillH1(4, x, energy);   // pair production
  }
}
```

```
// RunAction.cc, constructor: histogram 2, 3, 4; x = log10(E0/MeV), -1 ... 7
  am->CreateH1("ioni", "ionisation",        16, -1., 7.);
  am->CreateH1("brem", "bremsstrahlung", 16, -1., 7.);
  am->CreateH1("pair", "pair production",     16, -1., 7.);
```

- **Process names:** Depend on the particle: eIoni / muIoni / hIoni; eBrem / muBrems / hBrems; muPairProd / hPairProd.
- **Not in the list:** CoulombScat, Decay, hadronic… are written nowhere. Three processes today.
- **x = log10 E0:** Dimensionless, so no unit label. 16 bins = half a decade each.
- **Weight = energy:** Bin content = total energy taken by that process.
- **Ids 2, 3, 4:** Defined after edep (0) and depth (1).

**Checkpoint:** Build; ./ders surec_e.mac, surec_mu.mac, surec_p.mac → three ROOT files (≈ 5 s each).

# Analysis code

**macro (× 3: e-, mu-, proton)**

```
# surec_mu.mac
/run/initialize
/gun/particle mu-
/analysis/setFileName surec_mu
/run/printProgress 1000
/run/beamOn 4000
```

**run**

```
source /opt/geant4/bin/geant4.sh
cp -r G4Ders G4Ders-bragg
cd G4Ders-bragg
mkdir build
cd build
cmake ../.; make -j4
./ders surec_e.mac
./ders surec_mu.mac
./ders surec_p.mac
root -l draw_surec.C
```

**draw_surec.C**

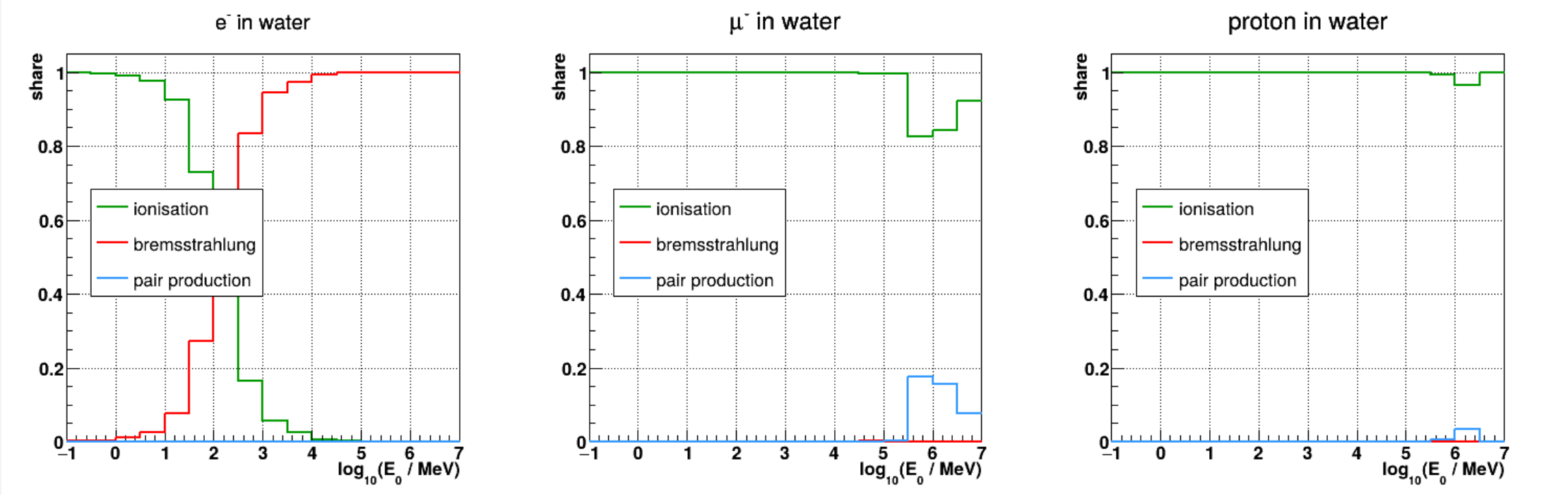
```
void panel(const char* file, const char* title)
{
  TFile* f = TFile::Open(file);
  TH1* ioni = (TH1*)f->Get("ioni");
  TH1* brem = (TH1*)f->Get("brem");
  TH1* pair = (TH1*)f->Get("pair");

  // total = sum of the three; share = process / total (bin by bin)
  TH1* total = (TH1*)ioni->Clone("total");
  total->Add(brem);  total->Add(pair);
  ioni->Divide(total);  brem->Divide(total);  pair->Divide(total);

  ioni->SetTitle(Form("%s;log10(E0 / MeV);share", title));
  ioni->Draw("hist");  brem->Draw("hist same");  pair->Draw("hist same");
}

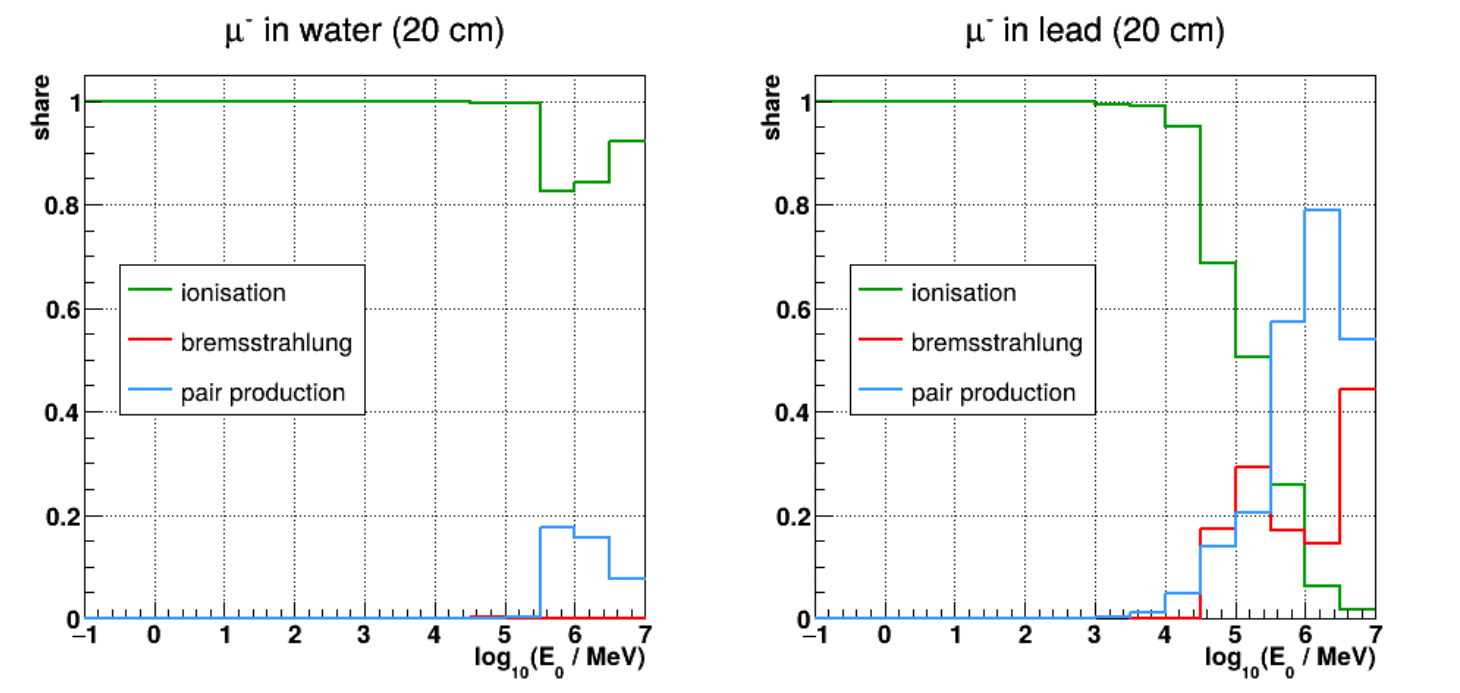
void draw_surec()
{
  gStyle->SetOptStat(0);
  TCanvas* c = new TCanvas("c", "", 1500, 500);
  c->Divide(3, 1);
  c->cd(1); gPad->SetGrid(); panel("surec_e.root",  "e^{-} in water");
  c->cd(2); gPad->SetGrid(); panel("surec_mu.root", "#mu^{-} in water");
  c->cd(3); gPad->SetGrid(); panel("surec_p.root",  "proton in water");
  c->SaveAs("surec.png");
}
```

# Result: Share of processes, in water




- **e⁻:** Ionisation = bremsstrahlung at ≈ 100 MeV (critical energy); above 1 GeV all radiation.  Proton: Ionisation everywhere.
- **μ⁻:** Pair production above 1 TeV, far below the tables: λ(pair) ≈ 3 m, λ(brems) ≈ 300 m at 10 TeV; rare in 20 cm. Lead: Next slide.

# Try: Same code, different material: G4_Pb




- **Single change:** FindOrBuildMaterial("G4_Pb"). Code, macro and histograms unchanged.
- **10 TeV μ⁻ in lead:** Radiative (pair + bremsstrahlung) ≈ 0.97, ionisation ≈ 0.03. In 20 cm of water: Pair ≈ 0.1, no bremsstrahlung.
- **Why:** Radiative cross sections ∝ $Z^2$; density 11×. λ(pair) ≈ 3 m → ≈ 6 cm. Critical energy: Water ≈ 1 TeV, Pb ≈ 140 GeV.
- **Caution:** For $e^-$ and protons a TeV shower develops fully in 20 cm of lead; minutes per event. Try it with muons.

**Lesson:** Monte Carlo sees the mean only if the process happens often enough. In short: Rare large losses need a thick or heavy target, or many events.

CODING WITH GEANT4 · TIPS

# Common mistakes

### Number without a unit — Units

G4Box("W", 10, 10, 10) → taken as 10 mm. Always write 10*cm.

### Giving the full length — Geometry

G4Box wants half-lengths: 150*um for a 300 µm thickness.

### No GeneratePrimaryVertex — Source

The event is empty, no error; the histogram stays empty.

### Command order — Macro

/gun/... commands come after /run/initialize.

### Histogram limit without a unit — Analysis

CreateH1(..., 0., 500., "keV") → 0–500 MeV! Limits in internal units: 500.*keV.

### CloseFile forgotten — Analysis

ROOT file empty or corrupt. Write + CloseFile in EndOfRunAction.

### Wrong volume name — Stepping

"silicon" ≠ "Silicon": No step is counted, the histogram is empty. The name must match the one in Construct.

### Wrong signature, no override — C++

UserSteppingAction(G4Step*) without const → Geant4 never calls it. override turns this into an error.

CLOSING

# Summary

| | |
|---|---|
| **1 · DetectorConstruction**<br>`Construct()` | vacuum world + 300 µm Si; solid, logical, physical volume; NIST material |
| **2 · PhysicsList**<br>`RegisterPhysics` | EM option4 + decay + hadronic; production cut |
| **3 · PrimaryGeneratorAction**<br>`GeneratePrimaries()` | G4ParticleGun: $\mu^-$ 1 GeV, +z, from in front of the slab; GeneratePrimaryVertex |
| **4 · SteppingAction**<br>`UserSteppingAction()` | in the slab? → fEvent->AddEdep(edep) |
| **5 · EventAction**<br>`Begin/EndOfEventAction()` | fEdep = 0 → FillH1(0, fEdep) |
| **6 · RunAction**<br>`Begin/EndOfRunAction()` | CreateH1("edep", ..., "keV"); OpenFile, Write, CloseFile |
| **7 · Bragg curve**<br>`4 files, +12 lines` | water 20 cm, proton 150 MeV; FillH1(1, z, edep) → dE/dx(z); peak at 15.7 cm |
| **8 · Share of processes**<br>`4 files, +25 lines` | random E0; GetCreatorProcess(); 3 histograms via if; share = process / total |

**Rules:** Measure in the step, sum in the event, write in the run. **Results:** Landau distribution, Bragg peak at 15.7 cm, critical energy of $e^-$ in water ≈ 100 MeV.